

Chapter 6

Results & Analysis

6.1 Implemented Results

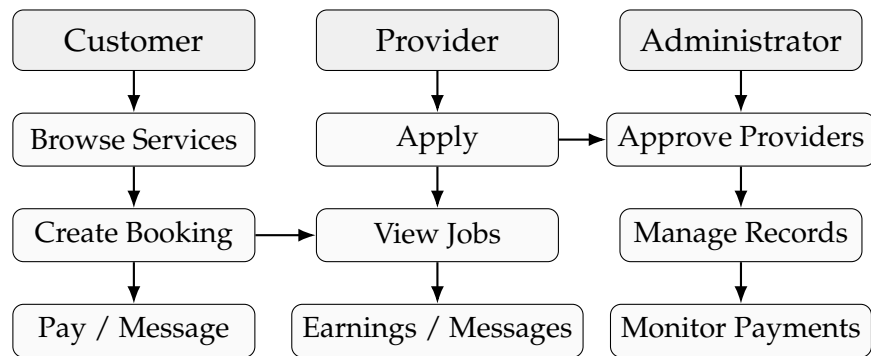
The final output is a working full-stack version of a home-service marketplace. I reviewed it from three sides because judging only the homepage would give an incomplete picture. A customer mainly sees service browsing and booking, but the provider and admin sides are what make Servico feel closer to a real service operation. My main contribution is visible most clearly in the service catalogue flow: categories and services can be managed from the admin side and then displayed dynamically on customer-facing pages.

From the customer side, the visible result starts from the homepage. A user can search services, browse categories, open service cards, view a service detail page, and move to booking. The important result for my module is that category and service data are not only static frontend content. When the admin creates or updates service data, the frontend can load that data through API calls and show it in service listings, category-wise pages, and detail pages. In the booking flow, the form keeps contact information, address, schedule, urgency, payment method, and price calculation together.

From the provider side, the result includes the application and dashboard flow. A provider can submit profile, experience, skill, availability, and document-related information. After approval, the provider side has dashboard, jobs, schedule, earnings, reviews, profile, profile picture display, message actions, and settings. I kept this part in the result chapter because the portal is not only for customers; service professionals also need a place to receive work and update job progress.

From the administrator side, Servico has the strongest control layer. Admin users can view dashboard values, manage users, approve or reject providers, manage services and uploaded images, manage categories, supervise bookings, moderate reviews, track transactions, manage promo codes, and update site settings. While checking this area, I focused most on category management and service management. I verified create, update, and delete behavior and checked whether changes were stored in PostgreSQL and reflected on the website. My repeated test flow was to create a category such as *Electrical Repair*, add services such as *AC Repair Service*, *Fan Repair Service*, or *Wiring Service*, check the API response, and then confirm that the

public service pages loaded the new records dynamically.



The final interface is divided into customer-facing pages and admin-management pages. I included the screens because the result is not only source code or API design. The screenshots show what a user actually sees and how the company side works with the same marketplace data.

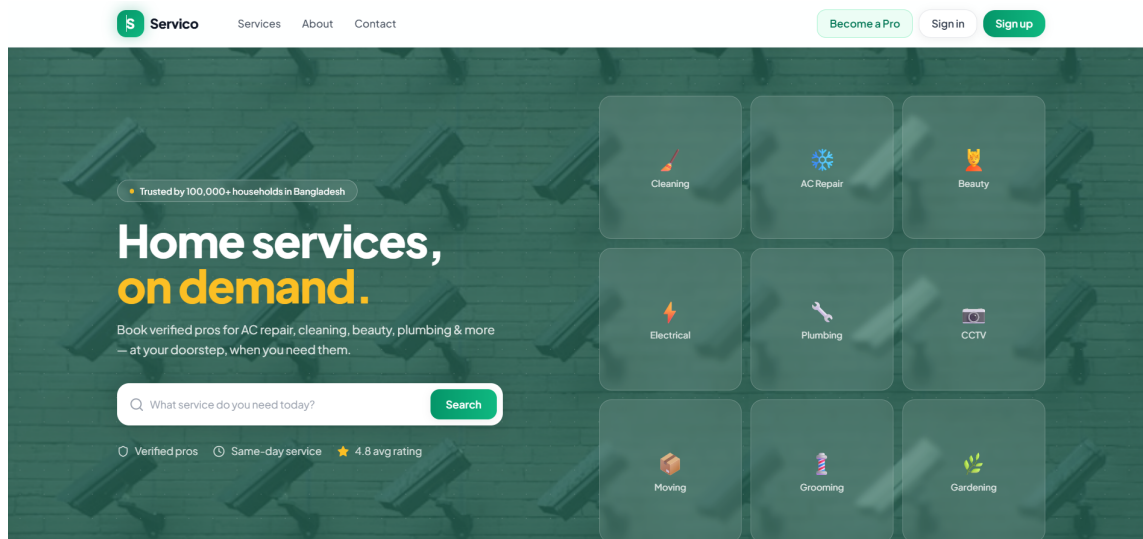


Figure 6.1: Customer homepage interface with service search and category navigation

Figure 6.1 shows the first screen of the customer side. The homepage presents the platform branding, service search, main categories, and options for sign in, sign up, and provider registration. This screen works as the starting point before a customer moves to the listing or booking flow.

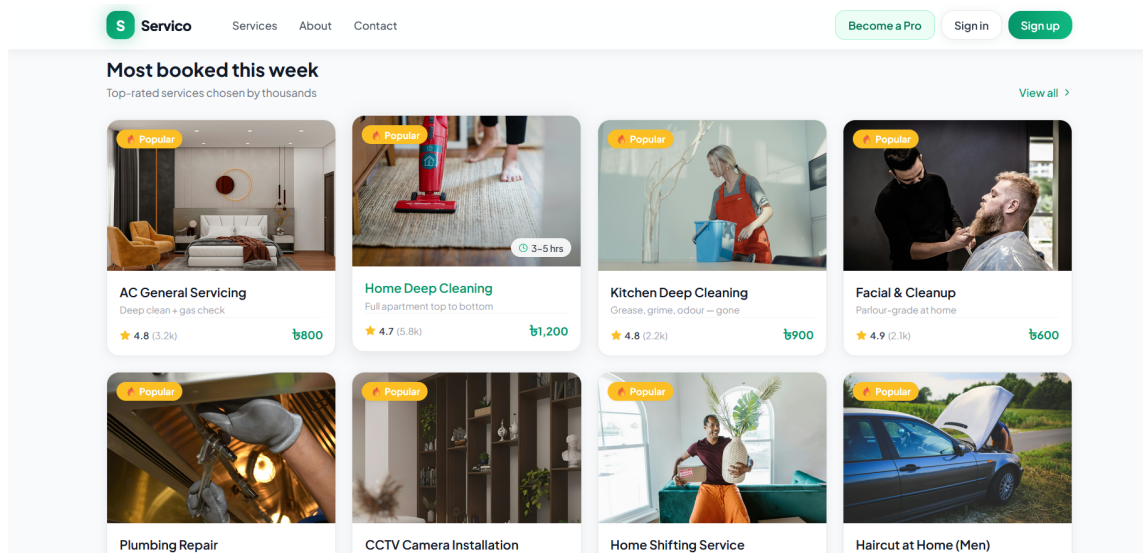


Figure 6.2: Customer service listing interface with popular service cards

Figure 6.2 shows the service listing result. Each card contains an image, title, short description, rating, popularity tag, and starting price. This screen is directly related to the service-management work because the list depends on category/service data coming from the backend instead of fixed frontend text. While testing this page, I checked search, sorting, category filtering, and the mapping between category slugs, category IDs, and service records.

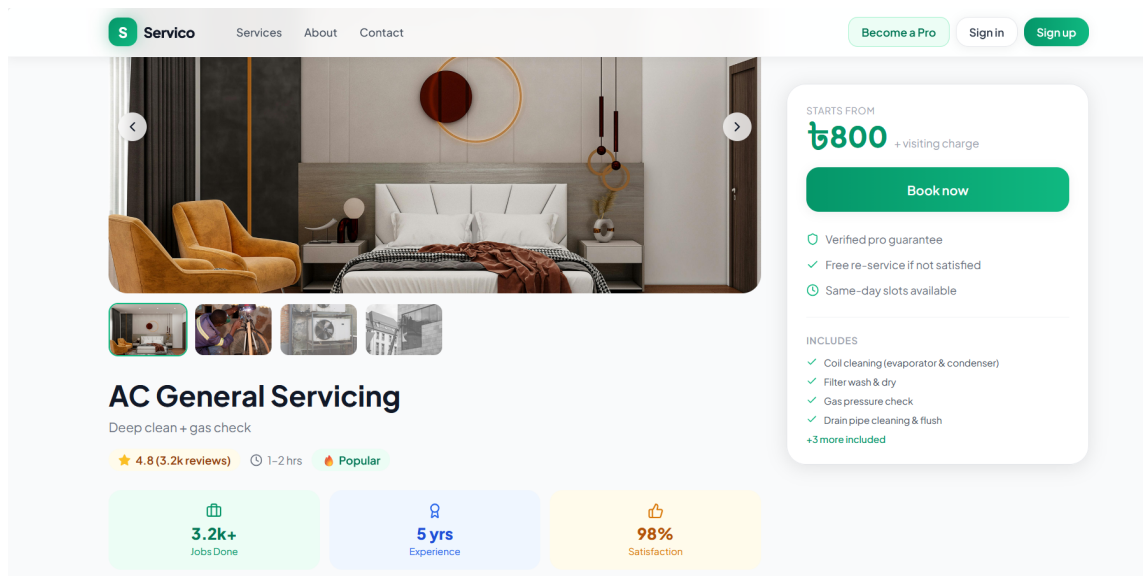


Figure 6.3: Service detail interface with price, included tasks, and booking action

Figure 6.3 shows the service detail page for AC General Servicing. This page brings together the service image, rating, duration, experience indicators, price, included work items, and the booking button. For my work, this screen is one of the main customer outputs because it connects service discovery with booking creation.

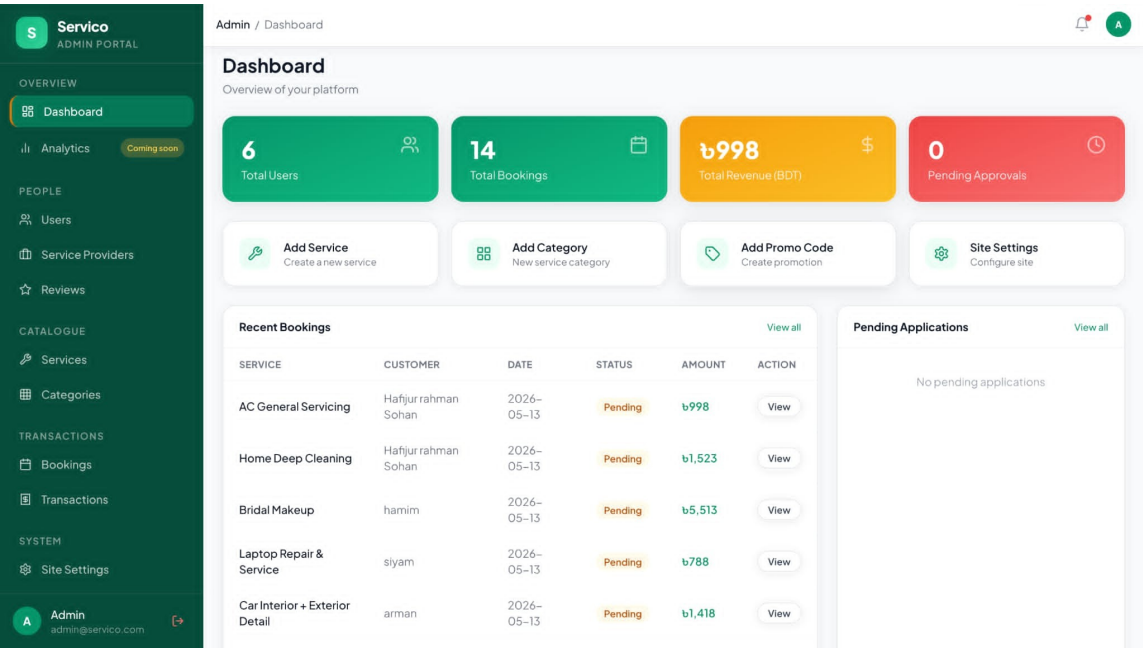


Figure 6.4: Admin dashboard overview with users, bookings, revenue, and pending approvals

Figure 6.4 shows the administrator dashboard. The dashboard summarizes total users, total bookings, total revenue, pending approvals, quick actions, recent bookings, and pending applications. This screen shows the operational side of the system, not only the customer-facing pages.

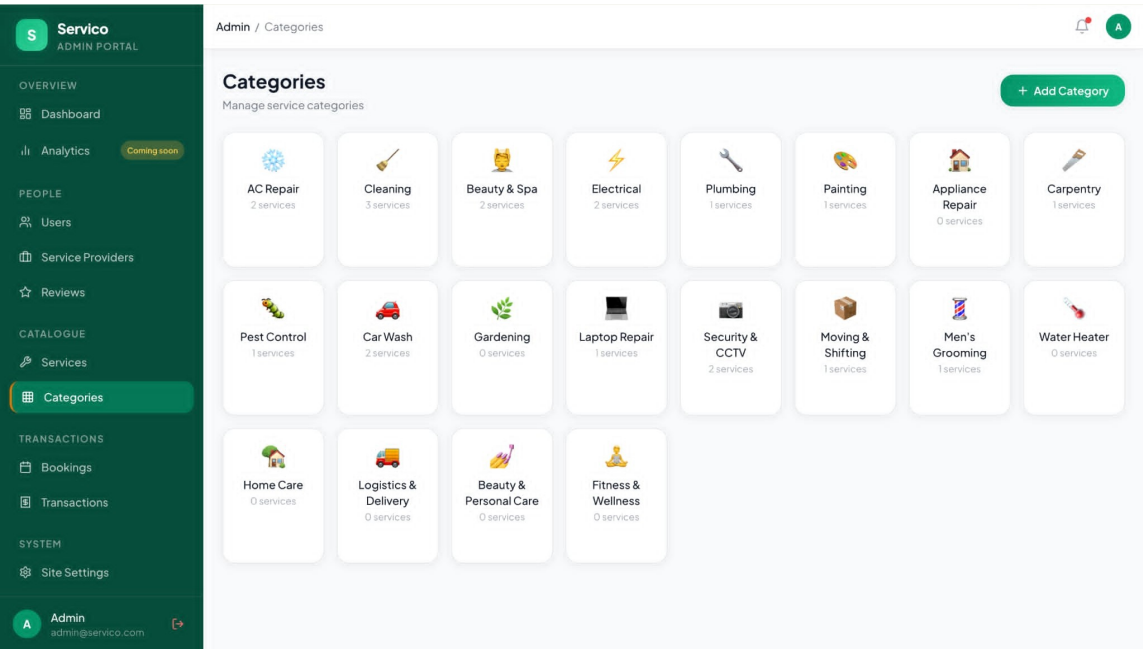


Figure 6.5: Admin category management interface for service catalogue control

Figure 6.5 shows category management from the admin portal. The categories are displayed with icons and service counts, and the admin can add a new category from the same page. This screen is one of the clearest outputs of my assigned work because it connects admin CRUD

operations with backend category records and frontend catalogue display. I tested the same idea from the service side as well: create or edit a service, upload an image, mark it as popular if needed, and confirm that the service list reflected the change.

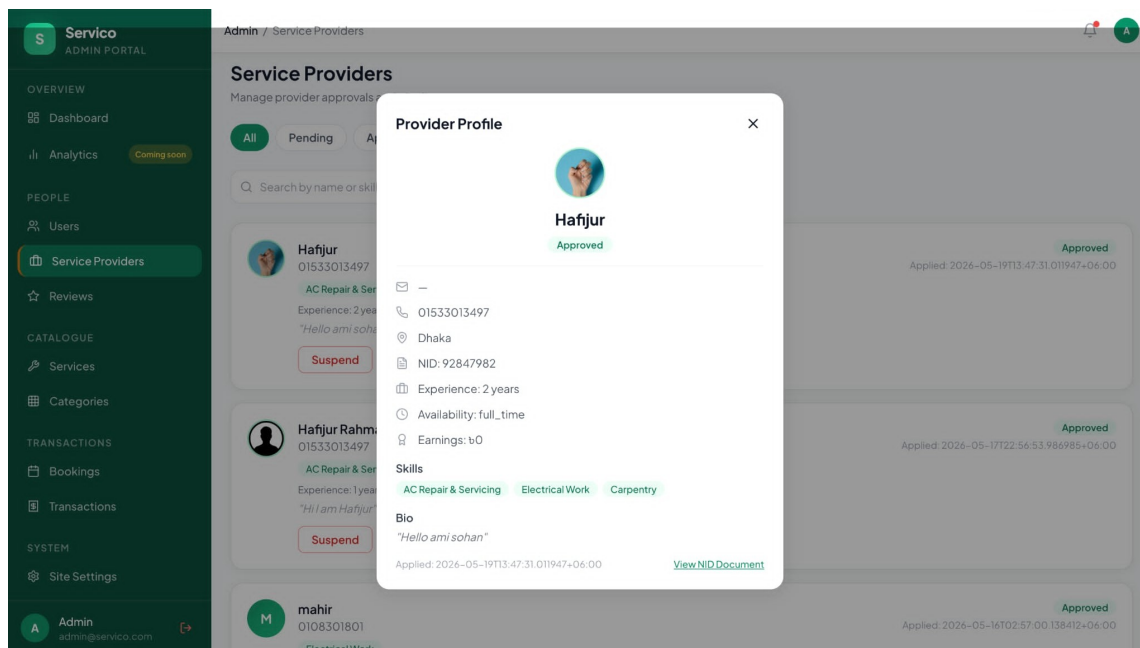


Figure 6.6: Admin service provider profile interface for provider review and supervision

Figure 6.6 shows the provider profile view inside the admin portal. The modal displays provider status, phone number, location, NID information, experience, availability, earnings, skills, bio, and document link. I included this screen because provider verification and supervision are part of the project boundary.

6.2 Backend Results

The backend result can be understood from the Django apps. The services app was the main backend area for my contribution because it stores categories, services, uploaded service images, FAQs, process steps, ratings, and included service items. I worked mainly around `models.py`, `serializers.py`, `views.py`, and `urls.py` in that app. I checked how these records are saved in PostgreSQL and returned to the React frontend. The image display issue also made me check serializer output carefully, because an uploaded image is not useful unless the API returns a URL that the React service card can display. The users app handles custom email-based users with role and status fields. The bookings app keeps customer, provider, schedule, price, status, and payment information. The chat app keeps booking-based messages with sender and read status. The providers, reviews, promos, and payments apps complete the rest of the marketplace flow.

This separation made the backend easier for me to follow. When I needed to understand a screen, I could usually trace it to a related route, view, serializer, or model instead of searching inside one large backend file. The central API prefix also made the React-to-Django

communication easier to understand.

6.3 Feature Coverage Analysis

6.4 Comparison with the Reference Platform

Sheba.xyz is already a mature service platform, while Servico is the version I studied and contributed to during internship. I used Sheba.xyz only as a reference to understand how a marketplace is usually arranged. Servico has similar broad ideas such as categories, customer booking, provider participation, admin supervision, booking communication, profile/service images, and local payment options. My work was mainly on the service/category management foundation, so I am not presenting it as a finished alternative to Sheba.xyz. A public service platform needs stronger dispatch logic, verified field operations, customer support, payment reconciliation, analytics, notifications, and mobile support.

6.5 Limitations

For viva, I would explain the limitations in a simple way: the service/category management flow works in the local project, and the core portal flows are present, but Servico is not ready for public service operation. Some frontend screens still need better loading, empty-state, and error handling when API data is missing. The payment flow was the most discussed unfinished feature in the team. The interfaces and payment-state ideas are present, but live merchant credentials, callback verification, full transaction storage, and accounting reconciliation are still missing. SMS notification, automatic provider assignment, production deployment, performance optimization, and stronger file-upload validation also need more work. These points show where my internship work ends and company-level operation begins.

Table 6.1: Feature coverage analysis

Feature Area	Coverage	Analysis
Service discovery	High	I verified categories, service listings, category-wise filtering, search behavior, sorting, popular services, and service detail pages from the UI and service APIs.
Service management	High	I tested category/service create, update, delete, image upload, popular flag update, PostgreSQL storage, API response structure, and frontend display synchronization from the admin dashboard.
Customer booking	High	Booking covers schedule, address, price calculation, urgent fee, VAT, promo/wallet fields, and payment method choice.
Booking communication	Medium to High	Customer-provider messages are tied to bookings, with polling, read state, and unread badges; real notification delivery is still incomplete.
Authentication	Medium to High	Login, registration, token storage, and protected routes work in the local version; security review is still needed before production.
Admin operations	High	Admin pages cover users, providers, services, service images, categories, bookings, reviews, transactions, promos, and settings.
Provider operations	Medium to High	Provider pages cover jobs, earnings, reviews, profile image display, and message actions; automatic dispatch is not completed.
Payments	Medium	Cash, bKash, Nagad, and card-state concepts are visible in records and UI flow, but live gateway processing, transaction verification, and reconciliation are not complete.
Testing	Medium	I manually checked service/category CRUD, service display, category filtering, and selected provider/admin flows; automated tests still need more work.

Chapter 7

Project as Engineering Problem Analysis

7.1 Sustainability of the Project/Work

When I looked at Servico as an engineering problem, my first question was whether another company developer could continue the work after me. The answer is mostly positive because the project uses common tools such as React, Django, and Django REST Framework. The frontend pages and backend apps are also separated by responsibility. In my service-management work, this separation was useful because I could trace a category or service from `AdminServicesPage.jsx`, to `src/lib/api.js`, to the Django service views and serializers, to the PostgreSQL record, and then back to `ServicesPage.jsx` or `CategoryPage.jsx`.

The business idea is also relevant in Bangladesh. People regularly need AC servicing, plumbing, electrical repair, cleaning, appliance repair, beauty care, and shifting support, but many of these services are still found through phone calls, personal contacts, or social media posts. A web portal cannot solve every field-service problem, but it can organize service discovery, booking records, provider approval, and admin monitoring in a better way. For long-term use, the company will need more than the current codebase. Deployment automation, monitoring, backup strategy, provider verification, analytics, and customer support tools have to be added. The service catalogue also has to stay updated, which means the admin category/service management flow must remain stable and easy to maintain. Provider quality must be checked, and payment settlement has to be reliable. From my understanding, these are company-operation issues as much as technical issues.

7.2 Social and Environmental Effects and Analysis

The social effect can be positive if the platform is managed carefully. Customers can look for household services from one place instead of depending only on personal contacts. Providers can receive jobs through a structured channel and build reputation through ratings and reviews. Administrators can see bookings, payments, provider approvals, and reviews from a central dashboard. This kind of structure can help small service providers who do not have

their own website or app.

There are also risks that I should not ignore. If provider approval rules are weak, a customer may get unsafe or low-quality service. If service information is wrong, customers may book the wrong category or misunderstand the price. If reviews are not moderated fairly, a provider's reputation can be damaged. If payment handling is not verified properly, both customer and provider can face problems. For that reason, provider screening, accurate service data, fair review handling, customer support, secure payment flow, and clear service rules are necessary before public launch.

The environmental effect is moderate. Digital records can reduce paper-based booking and manual coordination. However, provider travel can still increase fuel use if jobs are assigned without considering location. A future dispatch system should use area or distance information so nearby jobs can be handled more efficiently.

7.3 Addressing Ethics and Ethical Issues

Servico handles sensitive information such as names, phone numbers, customer addresses, provider NID-related data, provider documents, profile images, payment transaction references, reviews, user roles, and account status. It also stores service images and service/category records that affect what customers see before booking. Because of that, ethics is not only a theory topic here. If the application is used publicly, the company has to protect user privacy, control access, collect only necessary data, keep service information accurate, and handle uploaded documents carefully.

The ethical points I found most important are:

- **Privacy:** Customer addresses and phone numbers should not be visible outside the booking workflow. Provider NID-related data and documents should be limited to authorized admin review.
- **Security:** Tokens, passwords, payment references, uploaded service images, and provider documents need stronger protection before public use.
- **Data accuracy:** Service names, category slugs, category labels, prices, images, and descriptions should be checked because customers make booking decisions from this information.
- **Fairness:** Provider approval, suspension, job assignment, and review moderation need written company rules.
- **Transparency:** Customers need to see the service price, VAT, urgent fee, visiting charge, and payment status before confirming a booking.
- **Accountability:** Later versions need audit records for sensitive admin actions such as payment updates, provider rejection, and review deletion.

To address these issues, later development should include role-based permission tests, safer storage for sensitive documents, audit logs for admin actions, stronger validation, secure payment webhooks, privacy policy pages, service-data review rules, and clear review moderation rules. In the service module, admin users should be responsible for checking

service names, category assignment, uploaded images, price values, and popular-service marking before making them visible to customers. These changes matter because Servico deals with real people, real addresses, real service providers, and real service information that customers may depend on before booking.

Chapter 8

Lesson Learned

8.1 Problems Faced During this Period

The first problem I faced was scope control. At the beginning, the idea sounded simple: build an online service portal for home services. After opening the code and following the screens, I saw that the work was much bigger. Provider verification, dispatch, mobile payments, customer support, reviews, refunds, chat support, and admin dashboards all connect with each other. For the report, I had to separate my internship contribution from the parts that still need company-level time and resources.

The second challenge was frontend and backend consistency. The frontend pages needed user data, service data, provider application data, booking records, payment states, images, and message data in a predictable format. In my service-management work, this problem appeared clearly when services were saved in the database but did not display correctly on the frontend. Category filtering also failed at one point because the category identifier and slug mapping were not aligned properly. This made me check API responses more carefully instead of only looking at page design.

The third challenge was role-based access control. A customer, a provider, and an administrator should not see the same dashboard or use the same permissions. This required protected routes on the frontend, authenticated API calls, and a consistent session structure. I learned that authentication is not only the login screen; it also decides what a user can do after logging in. The fourth challenge was payment-flow complexity. bKash and Nagad are familiar payment methods in Bangladesh, but a real gateway integration needs merchant approval, callback verification, transaction identifiers, and reconciliation. In my work, I treated this as payment-flow and payment-state handling, not as a completed live settlement system.

Another challenge was service image handling. Uploading an image from the admin service form was not enough by itself. The serializer also had to return a usable image URL, and the React service card had to use that value correctly. This was a small issue on the screen, but it taught me how frontend display problems can come from backend response design.

The final challenge was documentation. The codebase has many modules, so the report had to organize requirements, design, implementation, testing, results, ethics, and later work without

making the explanation disconnected. Preparing the report forced me to check whether each point matched the actual files, screenshots, and database models.

8.2 Solution of those Problems

I handled the scope problem by dividing the work into customer, provider, and admin modules. After that, I selected the core flow for each module and avoided claiming that every production feature was completed. This made the report easier to defend in viva.

For frontend-backend consistency, I followed the centralized API functions in `src/lib/api.js` and authentication state in `src/lib/AuthContext.jsx`. For my assigned module, I compared `ServicesPage.jsx`, `CategoryPage.jsx`, and `AdminServicesPage.jsx` with the Django files under `backend/services/`. This made the code easier to follow because most requests and token handling were not scattered across many pages.

Role-based routing was handled through protected route components in `src/App.jsx`. Admin pages are guarded by admin checks, and provider pages are guarded by provider and provider-application checks. On the backend, JWT authentication protects private endpoints. For the payment flow, I first focused on payment models and endpoint structure. Then I worked with frontend modal flows for bKash and Nagad. This gave the application a local payment path while keeping full gateway hardening outside my completed internship scope. My mentor guided me especially on Django REST Framework serializers, RESTful API structure, model relationships, and how the React frontend should consume backend responses. One important lesson from that guidance was that service data should be managed dynamically from the dashboard instead of being hardcoded inside frontend components. For documentation, I followed the internship report template and used the sample report as a structural reference. I also kept the report tied to my actual codebase so that the diagrams, screenshots, testing discussion, and limitations could be explained during viva.

Chapter 9

Future Work & Conclusion

9.1 Future Works

The current version gives a working base, but I would still not call it production-ready. After reviewing the code, UI, screenshots, and testing flow, I think the next work should focus on the parts that matter when real customers and providers start using the portal:

- Move the React frontend and Django backend from local setup to a production setup with environment variables, SSL, domain configuration, database backup, and basic monitoring.
- Add automated tests for the flows I checked manually, especially category CRUD, service CRUD, service image upload, service loading, category filtering, booking, provider application, admin approval, payment state, chat, and reviews.
- Complete live bKash/Nagad gateway work with secure callbacks, transaction verification, refund handling, and reconciliation. This was the most discussed unfinished feature in the team because payment interfaces alone are not enough for real operation.
- Add SMS, e-mail, or push notifications for booking confirmation, provider assignment, payment update, and job completion.
- Improve booking chat with real-time message delivery, moderation rules, attachment rules, and notification support.
- Improve provider assignment by using service category, location, availability, rating, workload, and customer preference.
- Improve the service-management module with stronger validation for category slugs, better image handling, clearer empty states, and admin confirmation before deleting services or categories.
- Improve the analytics dashboard so admins can see service/category performance, popular services, booking trends, and provider workload more clearly.
- Add map-based address selection so customer location and provider travel can be handled more accurately.
- Add audit logs for sensitive admin actions such as payment update, provider approval, provider rejection, user suspension, and review moderation.
- Strengthen provider verification with document validation, approval notes, and

company-level background checks.

- Add customer support tickets, cancellation policy, refund rules, and dispute handling because field-service problems cannot be solved only by a booking form.
- Build a mobile-friendly PWA or mobile app because many customers and providers may prefer phone-based access.

9.2 Conclusion

This internship gave me practical experience with Servico, a full-stack online service portal for the Bangladesh home-services market. The report is based on my project review, code screenshots, UI screenshots, diagrams, and the parts I worked on during the internship. My main contribution was the Service Management Module, including category management, service management, frontend-backend API integration, PostgreSQL-backed CRUD operations, service listing, category-wise filtering, search/sorting, image upload handling, and dynamic service display. In practical terms, I worked around files such as `ServicesPage.jsx`, `CategoryPage.jsx`, `AdminServicesPage.jsx`, `src/lib/api.js`, and the Django service app. Servico also includes booking, booking-based messages, provider application and dashboard flow, image handling, transaction management, reviews, promo features, and admin management. The frontend uses React, Vite, React Router, and Tailwind CSS. The backend uses Django, Django REST Framework, JWT authentication, and separate Django apps.

The biggest learning for me was that a simple-looking web page depends on many backend decisions. For example, a service card on the frontend depends on category records, service fields, image data, API response mapping, and database storage. During development, I faced real problems such as services not displaying after database insertion, category filtering not matching the correct service records, and uploaded service images not always appearing in React components. Solving those issues helped me understand serializers, API response structure, category IDs, slugs, and frontend mapping more clearly. I also learned that a large marketplace project should not be presented as one intern's complete product. My contribution was connected mainly with service/category management and selected frontend-backend integration work, while other team members worked on authentication, provider management, booking, payment, review, and additional admin features.

Overall, I consider Servico a useful internship project and a strong base for future development. It is not a final commercial platform yet, and I kept that boundary clear throughout the report. The service-management flow is functional in the local project, but production deployment, automated testing, live payment verification, SMS notification, automatic provider assignment, monitoring, provider verification, customer support, and operational maintenance still need more company-level work. My mentor's guidance on Django REST Framework serializers, RESTful API design, database relationships, and React-to-Django integration helped me understand why a dynamic service management system is better than hardcoded service cards. This makes the report easier to defend because it matches the actual codebase and my internship role.

Bibliography

- [1] Sheba Platform Limited, "Sheba.xyz official website." Available: <https://www.sheba.xyz/>. Accessed: June 2, 2026.
- [2] Urban Company, "Urban Company official website." Available: <https://www.urbancompany.com/>. Accessed: June 2, 2026.
- [3] Meta Open Source, "React documentation." Available: <https://react.dev/>. Accessed: June 2, 2026.
- [4] Remix Software, Inc., "React router documentation." Available: <https://reactrouter.com/>. Accessed: June 2, 2026.
- [5] Vite Contributors, "Vite documentation." Available: <https://vite.dev/guide/>. Accessed: June 2, 2026.
- [6] Tailwind Labs Inc., "Tailwind css documentation." Available: <https://tailwindcss.com/docs>. Accessed: June 2, 2026.
- [7] Django Software Foundation, "Django documentation." Available: <https://docs.djangoproject.com/>. Accessed: June 2, 2026.
- [8] Encode OSS Ltd., "Django rest framework documentation." Available: <https://www.django-rest-framework.org/>. Accessed: June 2, 2026.
- [9] Jazzband, "Simple jwt documentation." Available: <https://django-rest-framework-simplejwt.readthedocs.io/>. Accessed: June 2, 2026.
- [10] M. Jones, J. Bradley, and N. Sakimura, "JSON Web Token (JWT)," RFC 7519, Internet Engineering Task Force, 2015. Accessed: June 2, 2026.
- [11] bKash Limited, "bKash official website." Available: <https://www.bkash.com/>. Accessed: June 2, 2026.
- [12] Nagad Limited, "Nagad official website." Available: <https://nagad.com.bd/>. Accessed: June 2, 2026.